If the market responds in time, the bet takes the odds from the response and matches them with the odds requested by the user. If they match, it creates a `MarketOddsAvailable`. No event is recorded; the actor sends this message to itself and processes it accordingly. Let's look at the command handler again to see how the actor processes this message:

```
object Bet {
  ...
  def handleCommands(...){
      ...
      case (state: OpenState, command: MarketOddsAvailable) =>
        validateMarket(state, command)
      ...
  }
  ...
}
```

The next stop is `validateMarket`, which is like all the other persists you've seen so far:

```
object Bet {
  ...

  private def validateMarket(
      state: OpenState,
      command: MarketOddsAvailable): Effect[Event, State] = {
    if (command.available) {                    ❶
      Effect.persist(MarketConfirmed(state))
    } else {                                    ❷
      Effect.persist(
        Failed(
          state.status.betId,
          s"market odds [${command.marketOdds}] not available"))
    }
  }
  ...
}
```

❶ If the market is available, the bet state is updated accordingly.

❷ If the market is not available, the bet gets into a FailedState.